JZ-HDL CHIP INFO SPECIFICATION

State: Beta — Version: 0.2.1

Contents

1. File Location and Naming

Chip data files are stored in `jz-hdl/data/` and are embedded into the compiler binary at build time. Files are named using the chip ID in lowercase with dashes, followed by `.json`:

`<family>-<size>-<package>-<speed>.json`

Examples: `gw2ar-18-qn88-c8-i7.json`, `ice40up-5k-sg.json`, `lfe5u-45f-6bg381.json`

2. Top-Level Structure

A chip data file is a single JSON object with the following top-level keys. When custom chip JSON is used, these top-level keys are validated strictly: required keys must be present, required value types must match, and unknown top-level keys are rejected.

Key	Type	Required	Description
`chipid`	string	Yes	Unique chip identifier
`description`	string	Yes	Human-readable description of the device
`boards`	array	Yes	Development boards featuring this chip
`resources`	object	Yes	Logic resource counts
`latches`	object	Yes	Latch capability description
`dsp`	object	Yes	DSP block description
`memory`	array	Yes	Memory types available on the device
`clock_gen`	array	Yes	Clock generation primitives
`differential`	object	No	Differential I/O support
`fixed_pins`	array	Yes	Pins with dedicated functions

3. `chipid`

A string identifying the device. Format: uppercase, dash-separated components encoding family, density, package, and speed grade.

`"chipid": "GW2AR-18-QN88-C8-I7"`

The compiler matches chip IDs case-insensitively by prefix. For example, `--chip-info GW2AR-18` matches all GW2AR-18 variants.

4. `boards`

An array of objects describing development boards that use this chip. May be empty. Each board entry must contain exactly `name` and `url`.

Key	Type	Required	Description
`name`	string	Yes	Board product name
`url`	string	Yes	URL for board info/docs

```
"boards": [
  {
    "name": "Tang Nano 20k",
```

```
    "url": "https://wiki.sipeed.com/hardware/en/tang/tang-nano-20k/nano-20k.html"
  }
]
```

5. `resources`

An object mapping resource names to integer counts. Keys are uppercase identifiers. Common resources:

Key	Description
`LUT<N>`	N-input lookup tables (e.g., `LUT4`, `LUT6`)
`DFF`	D flip-flops
`ALU`	Arithmetic logic units (Gowin only)
`IOB`	I/O blocks (user-available pins)
`LVDS_PAIRS`	LVDS differential pair count
`DIFF_PAIRS`	Generic differential pair count
`GSR`	Global set/reset

Not all keys are present on every device. Only include resources that exist on the chip.

```
"resources": {
  "LUT4": 20736,
  "DFF": 15552,
  "ALU": 15552,
  "IOB": 66,
  "LVDS_PAIRS": 22,
  "GSR": 1
}
```

6. `latches`

An object describing latch support in different logic blocks. Contains a `source` key plus one or more named block entries.

Key	Type	Required	Description
`source`	string	Yes	Datasheet reference for latch information

Each named block (`FAB` for fabric logic, `IOB` for I/O blocks) is an object:

Key	Type	Required	Description
`description`	string	Yes	What this logic block is
`D`	boolean	Yes	Whether D-latch is supported
`SR`	boolean	Yes	Whether SR-latch is supported
`modes`	array	No	Latch mode names (e.g., `["Transparent", "Gated"]`)

Key	Type	Required	Description
`note`	string	No	Additional information

```
"latches": {
  "source": "DS226-2.6E, Section 2.1 (CFU Architecture)",
  "FAB": {
    "description": "Configurable Function Unit (General Logic)",
    "D": false,
    "SR": false,
    "note": "Fabric storage elements are strictly edge-triggered registers."
  },
  "IOB": {
    "description": "I/O Blocks (Input/Output Pins)",
    "D": true,
    "SR": false,
    "modes": ["Transparent", "Gated"],
    "note": "Registers in I/O logic can be programmed as latches."
  }
}
```

7. `dsp`

An object describing DSP hardware. Contains metadata keys plus one or more named DSP type entries.

Key	Type	Required	Description
`source`	string	Yes	Datasheet reference
`description`	string	Yes	Overview of DSP capabilities

Each named DSP type (e.g., `MULT18X18`, `ALU54D`, `MAC16`) is an object:

Key	Type	Required	Description
`quantity`	integer	Yes	Number of instances on chip
`description`	string	Yes	What this DSP block does

```
"dsp": {
  "source": "DS226-2.6E, Table 1-1 p.2",
  "description": "Hardware multipliers and ALUs",
  "MULT18X18": {
    "quantity": 48,
    "description": "18x18 bit multiplier"
  }
}
```

8. `memory`

An array of memory type objects. Each object describes one category of memory available on the chip.

8.1 Common Fields

Key	Type	Required	Description
`type`	string	Yes	Memory type (see Section 8.2)
`source`	string	Yes	Datasheet reference

8.2 Memory Types

The `type` field must be one of:

- `"SDRAM"` - Embedded SDRAM (on-chip or in-package)
- `"DISTRIBUTED"` - Distributed memory from logic slices
- `"BLOCK"` - Block RAM (BSRAM, EBR)
- `"SPRAM"` - Single-port RAM
- `"FLASH"` - User flash memory

8.3 SDRAM Type

Key	Type	Required	Description
`type`	string	Yes	`"SDRAM"`
`source`	string	Yes	Datasheet reference
`capacity_mbits`	integer	Yes	Total capacity in megabits
`bus_width`	integer	Yes	Data bus width in bits
`max_freq_mhz`	number	Yes	Maximum operating frequency (MHz)

```
{
  "type": "SDRAM",
  "source": "DS226-2.6E, Table 1-2 p.3",
  "capacity_mbits": 64,
  "bus_width": 32,
  "max_freq_mhz": 166
}
```

8.4 DISTRIBUTED Type

Key	Type	Required	Description
`type`	string	Yes	`"DISTRIBUTED"`
`source`	string	Yes	Datasheet reference
`description`	string	Yes	Human-readable description
`total_bits`	integer	Yes	Total distributed memory capacity in bits

Key	Type	Required	Description
`r_ports`	integer	Yes	Number of read ports
`w_ports`	integer	Yes	Number of write ports
`configurations`	array	Yes	Available width/depth configurations

Each configuration object:

Key	Type	Required	Description
`width`	integer	Yes	Data width in bits
`depth`	integer	Yes	Number of entries

```
{
  "type": "DISTRIBUTED",
  "source": "DS226-2.6E, Table 1-1 p.2",
  "description": "Distributed memory from CFU slices",
  "total_bits": 40960,
  "r_ports": 1,
  "w_ports": 1,
  "configurations": [
    { "width": 1,  "depth": 40960 },
    { "width": 2,  "depth": 20480 },
    { "width": 4,  "depth": 10240 },
    { "width": 8,  "depth": 5120 },
    { "width": 16, "depth": 2560 }
  ]
}
```

8.5 BLOCK, SPRAM, and FLASH Types

These types share a common structure for block-based memory:

Key	Type	Required	Description
`type`	string	Yes	`"BLOCK"`, `"SPRAM"`, or `"FLASH"`
`source`	string	Yes	Datasheet reference
`description`	string	Yes	Human-readable description
`quantity`	integer	Yes	Number of memory blocks
`bits_per_block`	integer	No	Bits per individual block
`total_bits`	integer	Yes	Total memory capacity in bits
`max_freq_mhz`	number	No	Maximum operating frequency (MHz)
`note`	string	No	Additional information
`modes`	array	Yes	Available operating modes

Mode Object

Key	Type	Required	Description
`name`	string	Yes	Mode name (e.g., “Single Port”)
`description`	string	No	Mode description
`ports`	array	Yes	Port definitions for this mode
`configurations`	array	Yes	Available width/depth configurations

Port Object

Key	Type	Required	Description
`id`	string	Yes	Port identifier (e.g., “A”, “B”)
`read`	boolean	Yes	Whether port supports reads
`write`	boolean	Yes	Whether port supports writes

Configuration Object

Key	Type	Required	Description
`width`	integer	Yes	Data width in bits
`depth`	integer	Yes	Number of entries

Mode Names The `name` field is a human-readable description of the mode, not a machine-parsed type. The `ports` array defines the actual read/write capabilities. Any descriptive name may be used; prefer the vendor’s terminology for the device. Common examples:

Name	Vendor	Typical Ports
`Single Port`	All	1 R/W port
`Dual Port`	Generic	2 independent R/W ports
`True Dual Port`	Lattice, Xilinx	2 independent R/W ports
`Simple Dual Port`	Xilinx	Write on Port A, read on Port B
`Semi-Dual Port`	Gowin	Write on Port A, read on Port B
`Pseudo-Dual Port`	Lattice	Write on Port A, read on Port B
`Read Only Memory`	All	1 read-only port
`Read Only`	All	1 read-only port (flash)


```
{
  "type": "BLOCK",
  "source": "DS226-2.6E, Table 2-5 p.19",
  "description": "Block Static Random Access Memory (BSRAM).",
  "quantity": 46,
  "bits_per_block": 18432,
  "total_bits": 847872,
  "max_freq_mhz": 380,
  "note": "46 blocks x 18,432 bits = 847,872 bits.",
```

```
  "modes": [{
    "name": "Single Port",
    "description": "Single port read/write at one clock edge",
    "ports": [
      { "id": "A", "read": true, "write": true }
    ],
    "configurations": [
      { "width": 1, "depth": 16384 },
      { "width": 2, "depth": 8192 }
    ]
  }]
}
```

9. `clock_gen`

An array of clock generator objects. Each describes one type of clock generation primitive (oscillator, PLL, clock divider).

9.1 Common Fields

Key	Type	Required	Description
`type`	string	Yes	Generator type: `"pll"`, `"pll2"`, `"dll"`, `"clkdiv"`, `"osc"`, `"buf"`, or numbered variants (e.g., `"clkdiv2"`, `"buf2"`)
`source`	string	Yes	Datasheet reference
`description`	string	Yes	Human-readable description
`count`	integer	Yes	Number of instances available on chip
`mode`	string	No	Operating mode (e.g., `"local"` for clkdiv)
`chaining`	boolean	No	Whether PLLs can be chained (PLL only)
`pad_exclusive`	boolean	No	Whether the PAD variant exclusively owns the IO cell on the reference clock pin (see Section 9.10)
`feedback_wire`	string	No	Base name for auto-generated feedback wire (see Section 9.9)
`map`	object	Cond.	Backend-specific instantiation templates. Required unless `variants` is present
`variants`	array	Cond.	Variant-dispatched backend templates (see Section 9.3). Required unless `map` is present
`parameters`	object	No	Configurable parameters
`outputs`	object	Yes	Clock output definitions
`inputs`	object	No	Clock input definitions (PLL only)
`derived`	object	No	Derived values computed from parameters
`constraints`	array	No	Validation constraints

Each `clock_gen` entry must have **exactly one** of `map` or `variants`. Entries whose primitive never changes shape use `map` as in previous revisions. Entries whose vendor exposes several distinct primitive cells for the same physical hardware use `variants` (see Section 9.3).

9.2 `map` Object

The `map` object provides backend-specific HDL templates for instantiating the clock primitive. Keys are backend names; values are arrays of strings that concatenate to form the complete instantiation code.

Currently supported backends: - `"verilog-2005"` - Verilog 2005 module instantiation - `"rtlil"` - Yosys RTLIL cell definition

Template strings use the `%%name%%` placeholder syntax (see Section 12).

```
"map": {
  "verilog-2005": [
    "OSCZ #(\n",
    "    .FREQ_DIV(%%DIV%%)\n",
    ") u_osc (\n",
    "    .OSCEN(1'b1),\n",
    "    .OSCOUT(%%BASE%%)\n",
    ");\n"
  ],
  "rtlil": [
    "cell \\OSCZ $auto$osc\n",
    "  parameter \\FREQ_DIV %%DIV%%\n",
    "  connect \\OSCEN 1'1\n",
    "  connect \\OSCOUT \\%%BASE%%\n",
    "end\n"
  ]
}
```

9.3 `variants` Array

Some clock generators are exposed by the vendor as multiple distinct primitive cells that all share the same physical hardware, where the choice of cell depends on how the user wires the generator. For example, the iCE40 PLL is a single physical block, but the vendor provides four primitive cells (`SB_PLL40_CORE`, `SB_PLL40_2F_CORE`, `SB_PLL40_PAD`, `SB_PLL40_2F_PAD`) chosen by whether the reference clock is driven from a pad or from fabric, and whether one or two outputs are used.

The `variants` array lets a single `clock_gen` entry describe all such cells as one logical generator. The compiler inspects how the user wired the generator and automatically selects the matching variant at elaboration time. The `count` field applies to the logical generator as a whole, so a chip with one physical PLL has `count: 1` regardless of how many variant cells it exposes.

A `clock_gen` entry must have **exactly one** of `map` or `variants`. When `variants` is present, the top-level `map` field must be omitted.

Each element of the `variants` array is an object:

Key	Type	Required	Description
`when`	object	Yes	Match conditions under which this variant is selected
`map`	object	Yes	Backend templates for this variant (same format as Section 9.2)

9.3.1 `when` Object The `when` object is a set of fact/value pairs. A variant is selected when **every** fact in its `when` matches the corresponding fact computed by the compiler for the user's instantiation.

Each key is a **fact name** drawn from a fixed, documented vocabulary. Unknown keys cause a chip-data load error. Each value is the expected value of that fact; value types are per-fact.

Fact vocabulary:

Fact	Value type	Description
`input.<NAME>.source`	`"pad"` | `"fabric"`	Where the signal wired to input `<NAME>` originates. `"pad"` means the signal traces back to a project-level input pin; `"fabric"` means it is driven by internal logic. This fact is binary: every signal in a JZ-HDL design is either a pad signal or a fabric signal.
`output.count`	integer	Number of `OUT`/`WIRE` declarations the user provided on this clock generator unit.

Additional facts may be added in future revisions as new chips require them. An `input.<NAME>.type` fact (values such as `"osc"`, `"clock"`, `"pin"`, …) is reserved for future use and is not yet implemented.

9.3.2 Variant Selection Rules When a `clock_gen` entry uses `variants`:

1. For each unit instantiation, the compiler computes the set of facts for that unit from the user's wiring and declarations.
2. It walks the `variants` array and finds every variant whose `when` matches all computed facts.
3. **Exactly one** variant must match. Zero matches or more than one match is a compile-time error reported against the user's `clock_gen` unit.

Additionally, at chip-data load time, the compiler verifies that the `variants` array is **exhaustive and disjoint** over the cartesian product of all fact values referenced anywhere in the entry's `when` clauses. A chip-data file whose variants leave a reachable fact tuple unmatched, or whose variants overlap, is rejected. This catches authoring errors in chip data rather than letting them surface as confusing user-facing diagnostics.

```
{
  "type": "pll",
  "source": "DS-ICE40-UltraPlus, Section: PLL",
  "description": "Phase-Locked Loop. The actual primitive cell is selected automatically based
  "count": 1,
  "inputs": {
    "REF_CLK": {
      "description": "Reference clock input to the PLL",
      "required": true,
      "width": 1,
      "requires_period": true,
      "min_mhz": 10,
      "max_mhz": 133
    }
  },
  "parameters": {
    "DIVR":            { "type": "int", "default": 0, "min": 0, "max": 15,  "description": "Re
    "DIVF":            { "type": "int", "default": 0, "min": 0, "max": 127, "description": "Fe
    "DIVQ":            { "type": "int", "default": 0, "min": 0, "max": 7,   "description": "Ou
    "FILTER_RANGE":    { "type": "int", "default": 1, "min": 0, "max": 7,   "description": "PL
    "PLLOUT_SELECT_A": { "type": "string", "default": "GENCLK", "valid": ["GENCLK", "GENCLK_HA
    "PLLOUT_SELECT_B": { "type": "string", "default": "GENCLK", "valid": ["GENCLK", "GENCLK_HA
  },
  "derived": {
    "FVCO": {
      "description": "Internal VCO frequency",
      "expr": "(1000.0 / REF_CLK_period_ns) * (DIVF + 1) / (DIVR + 1)",
      "min": 533,
      "max": 1066
    }
  },
  "outputs": {
    "BASE_A": { "description": "Primary PLL output",   "port": "PLLOUTGLOBALA", "is_clock": tr
    "BASE_B": { "description": "Secondary PLL output", "port": "PLLOUTGLOBALB", "is_clock": tr
    "LOCK":   { "description": "PLL lock indicator",   "port": "LOCK",          "is_clock": fa
  },
  "variants": [
    {
      "when": { "input.REF_CLK.source": "pad", "output.count": 1 },
      "map": {
        "verilog-2005": [ "SB_PLL40_PAD #(\n  ...\n) u_pll (...);\n" ],
        "rtlil":        [ "cell \\SB_PLL40_PAD ...\nend\n" ]
      }
    },
    {
      "when": { "input.REF_CLK.source": "pad", "output.count": 2 },
```

```
      "map": {
        "verilog-2005": [ "SB_PLL40_2F_PAD #(\n  ...\n) u_pll (...);\n" ],
        "rtlil":        [ "cell \\SB_PLL40_2F_PAD ...\nend\n" ]
      }
    },
    {
      "when": { "input.REF_CLK.source": "fabric", "output.count": 1 },
      "map": {
        "verilog-2005": [ "SB_PLL40_CORE #(\n  ...\n) u_pll (...);\n" ],
        "rtlil":        [ "cell \\SB_PLL40_CORE ...\nend\n" ]
      }
    },
    {
      "when": { "input.REF_CLK.source": "fabric", "output.count": 2 },
      "map": {
        "verilog-2005": [ "SB_PLL40_2F_CORE #(\n  ...\n) u_pll (...);\n" ],
        "rtlil":        [ "cell \\SB_PLL40_2F_CORE ...\nend\n" ]
      }
    }
  ]
}
```

9.3.3 Example In this example the four variants cover the full 2×2 cartesian product of `(pad|fabric) × (1|2)`, so the exhaustive/disjoint check passes. The user writes a single `pll` unit in their JZ-HDL source and the compiler picks the correct primitive cell at elaboration.

9.4 `parameters` Object

An object mapping parameter names to their definitions. Each parameter:

Key	Type	Required	Description
`description`	string	Yes	What this parameter controls
`type`	string	Yes	`"int"`, `"double"`, or `"string"`
`default`	number or string	Yes	Default value
`min`	number	No	Minimum value (for `"int"` or `"double"` with range)
`max`	number	No	Maximum value (for `"int"` or `"double"` with range)
`valid`	array	No	Enumerated list of valid values

A parameter uses either `min`/`max` (inclusive range) or `valid` (enumerated set), not both.

```
"parameters": {
  "DIV": {
    "description": "Oscillator divider. Base frequency is ~250MHz divided by DIV.",
    "type": "int",
    "default": 2,
```

```
    "valid": [2, 4, 6, 8, 10, 12, 14, 16]
  },
  "ODIV": {
    "description": "Output divider select",
    "type": "int",
    "default": 8,
    "valid": [2, 4, 8, 16, 32, 48, 64, 80, 96, 112, 128]
  }
}
```

9.5 `inputs` Object

An object mapping input names to their definitions. Each key is the input name used in the JZ-HDL `IN <input_name> <signal>` syntax. Used for clock generators that require external signals (reference clocks, enables, etc.).

Key	Type	Required	Description
`description`	string	Yes	What this input is
`required`	boolean	No	Whether this input must be provided; defaults to `true`. If `default` is provided, `required` is implicitly `false`
`default`	string	No	Default value used when the input is omitted (e.g., `"1'b1"`). When present, `required` is `false`
`width`	integer	No	Signal width in bits (required for connected inputs)
`requires_period`	boolean	No	Whether the compiler needs the clock period for this input. Only set for clock-type inputs like `REF_CLK`
`min_mhz`	number	No	Minimum input frequency in MHz (only for clock-type inputs)
`max_mhz`	number	No	Maximum input frequency in MHz (only for clock-type inputs)

Required clock input (PLL reference clock):

```
"inputs": {
  "REF_CLK": {
    "description": "Reference clock input to the PLL",
    "required": true,
    "width": 1,
    "requires_period": true,
    "min_mhz": 3,
    "max_mhz": 500
  }
}
```

Optional input with default (clock enable):

```
"inputs": {
  "REF_CLK": {
    "description": "Reference clock input to the buffer",
    "required": true,
    "width": 1,
    "requires_period": true
  },
  "CE": {
    "description": "Clock enable (active-high). When deasserted, output is held low.",
    "required": false,
    "default": "1'b1",
    "width": 1
  }
}
```

When `required` is `false` and `default` is provided, the JZ-HDL `IN` line for that input may be omitted. The compiler substitutes the `default` value in the instantiation template.

No-input generator (OSC):

```
"inputs": {
  "REF_CLK": {
    "required": false,
    "description": "OSC has no external clock input"
  }
}
```

9.6 `derived` Object

An object mapping derived value names to their definitions. Derived values are computed from parameters and inputs and used in output frequency expressions and constraint checking.

Key	Type	Required	Description
`description`	string	No	What this derived value represents
`expr`	string	Yes	Expression to compute the value (see Section 11)
`min`	number	No	Minimum valid value for range checking
`max`	number	No	Maximum valid value for range checking

```
"derived": {
  "FVCO": {
    "description": "Internal VCO frequency",
    "expr": "(1000.0 / refclk_period_ns) * (FBDIV + 1) * ODIV / (IDIV + 1)",
    "min": 500,
    "max": 1250
  },
  "PSDA_SEL": {
    "description": "Binary string for PHASESEL",
    "expr": "toString(PHASESEL * 2, BIN, 4)"
```

```
  }
}
```

9.7 `outputs` Object

An object mapping output names to their definitions. Each output represents either a clock signal or a status signal produced by the generator.

Key	Type	Required	Description
`description`	string	No	What this output is
`port`	string	Yes	Physical port name on the primitive
`is_clock`	boolean	Yes	`true` if this output is a clock signal, `false` if it is a status/control signal (e.g., LOCK)
`frequency_mhz`	object	No	Frequency specification (required when `is_clock` is `true`)
`phase_deg`	object	No	Phase specification

The `is_clock` field distinguishes clock outputs (which must be declared in the `CLOCKS` block and have a computable frequency) from non-clock outputs (such as PLL lock indicators) which are status signals and do not represent clocks.

The `frequency_mhz` object:

Key	Type	Required	Description
`description`	string	No	Human-readable frequency description
`expr`	string	Yes	Expression computing frequency in MHz (see Section 11)

The `phase_deg` object:

Key	Type	Required	Description
`description`	string	No	Human-readable phase description
`expr`	string	Yes	Expression computing phase in degrees

Every clock generator must have at least a `BASE` output. The `BASE` output is the primary clock output.

Non-clock outputs (e.g., `LOCK`) set `is_clock` to `false` and omit `frequency_mhz`:

```
"LOCK": {
  "description": "PLL lock indicator, high when PLL is locked",
  "port": "LOCK",
  "is_clock": false
}
```

9.8 `constraints` Array

An array of constraint objects for validation.

Key	Type	Required	Description
`description`	string	Yes	Human-readable constraint description
`rule`	string	Yes	Machine-readable rule identifier

```
"constraints": [
  {
    "description": "VCO frequency must be within 500-1250 MHz range.",
    "rule": "FVCO range check"
  }
]
```

9.9 `feedback_wire` Field

Some clock generators (PLLs, MMCMs) require an internal feedback path where the feedback output must be wired back to the feedback input. This is hardware plumbing that the user never configures or connects — the compiler handles it automatically.

The `feedback_wire` field specifies the base name for the auto-generated feedback wire. When present, the compiler:

1. Declares a wire named `<feedback_wire>_<cg_index>_<unit_index>` (e.g., `clkfb_0_0`)
2. Makes the base name available as a template placeholder (e.g., `%%clkfb%%`)

The placeholder resolves to the full wire name including indices, ensuring unique feedback wires when multiple clock generators are instantiated.

```
{
  "type": "pll",
  "feedback_wire": "clkfb",
  "map": {
    "verilog-2005": [
      "PLLE2_BASE #(\n",
      "    ...\n",
      ") u_pll_%%instance_idx%% (\n",
      "    .CLKIN1(%%REF_CLK%%),\n",
      "    .CLKFBIN(%%clkfb%%),\n",
      "    .CLKFBOUT(%%clkfb%%),\n",
      "    .CLKOUT0(%%BASE%%),\n",
      "    ...\n",
      ");\n"
    ]
  }
}
```

In this example, for the first PLL unit in the first clock gen block, `%%clkfb%%` resolves to `clkfb_0_0`, producing:

```
wire clkfb_0_0;
PLLE2_BASE #(...) u_pll_0_0 (
    .CLKFBIN(clkfb_0_0),
    .CLKFBOUT(clkfb_0_0),
    ...
);
```

If `feedback_wire` is omitted, no feedback wire is generated and the placeholder is not available.

9.10 `pad_exclusive`

When `true`, indicates that the PAD variant of this clock generator exclusively consumes the IO cell on the reference clock pin. This is relevant for architectures (such as iCE40) where the PLL's PAD primitive and the IO buffer (e.g., `SB_IO`) share the same physical BEL and cannot coexist.

When this flag is set and the compiler selects a PAD variant (i.e., the template contains `PACKAGEPIN`) **and** the reference clock pin is also bound to a top-module port, the compiler:

1. Removes the pin from the wrapper module's port list (preventing the synthesis tool from inserting an IO buffer).
2. Declares the pin as an internal `wire` instead. The PLL's `PACKAGEPIN` (which is `inout`) connects this wire to the physical pad.
3. Omits the pin from PCF constraint output.
4. Uses `get_nets` instead of `get_ports` for SDC clock constraints referencing that pin.

The reference clock signal reaches fabric logic through the GBIN (Global Buffer Input) path, which is a dedicated pad-to-global-network route that does not conflict with the PLL.

If `pad_exclusive` is `false` or omitted, no special handling is applied.

10. `differential`

An object describing differential I/O support. Optional; omit if the device has no differential I/O. When present, the object is validated strictly; unknown keys and malformed nested objects are rejected.

10.1 Top-Level Fields

Key	Type	Required	Description
`source`	string	Yes	Datasheet reference
`type`	string	Yes	`"true"` (native LVDS) or `"emulated"` (GPIO-based)
`io_type`	string	Yes	I/O standard name (e.g., `"LVDS25"`, `"LVCMOS33D"`)
`output`	object	No	Output differential support
`input`	object	No	Input differential support
`clock`	object	No	Differential clock input support (see §10.3)

At least one of `output` or `input` must be present. Some chips support only differential output or only differential input.

10.2 `output` / `input` Objects

Each contains:

Key	Type	Required	Description
`buffer`	object	Yes	Differential buffer primitive
`serializer`	array	No	Output serializer options (output only)
`deserializer`	array	No	Input deserializer options (input only)

Buffer Object

Key	Type	Required	Description
`description`	string	Yes	What this buffer does
`map`	object	Yes	Backend templates (same format as clock_gen map)

Serializer / Deserializer Array An array of serializer (or deserializer) option objects, ordered by ascending ratio. Each chip lists all the serialization ratios it supports. For a requested differential data width `N`, the compiler selects the smallest ratio that is greater than or equal to `N`; the requested width does not need to exactly match a listed ratio. If the selected primitive ratio is wider than `N`, backend wrapper generation and/or the emitted template must pad, tie off, or otherwise safely handle the surplus lanes without changing the logical project-visible width. If no listed ratio is large enough, backend code generation reports an unsupported-width compile error.

Each element:

Key	Type	Required	Description
`description`	string	Yes	What this primitive does
`ratio`	integer	Yes	Serialization/deserialization ratio
`required_clocks`	array	Yes	Clock/reset signals the primitive needs; subset of `["fclk", "pclk", "reset"]`
`map`	object	Yes	Backend templates

The `required_clocks` array tells the compiler which pin attributes (`fclk`, `pclk`, `reset`) are mandatory for pins that use the selected serializer or deserializer primitive. For example, a simple DDR primitive (ratio 2) may only require `["fclk"]`, while a 10:1 serializer typically requires `["fclk", "pclk", "reset"]`. The compiler uses this field to validate differential pin declarations in the project file, rather than hardcoding which attributes are required.

10.3 `clock` Object

Optional. When present, the compiler uses this buffer primitive instead of `input.buffer` for differential pins that drive clock domains. On some architectures (e.g., Xilinx 7-series) the clock input buffer routes directly to the global clock network (`IBUFGDS`) rather than through general fabric interconnect (`IBUFDS`). If `clock` is omitted, clock inputs use the regular `input.buffer`.

Key	Type	Required	Description
`buffer`	object	Yes	Differential clock buffer primitive

The `buffer` object has the same structure as `input.buffer` / `output.buffer` (a `description` string and a `map` object with backend templates).

10.4 Serializer / Deserializer Arrays

When a chip supports multiple ratios using different hardware primitives (e.g., Gowin OSER4, OSER8, OSER10), each primitive gets its own entry with its own template. When a wider ratio requires cascading hardware (e.g., Xilinx OSERDESE2 master+slave for 10:1), the template is self-contained — it emits all necessary wire declarations and primitive instantiations. The compiler does not need to know whether a template uses one primitive or multiple; it simply picks the best-fit ratio and emits the template. The selected primitive may be wider than the requested logical width; any surplus serializer/deserializer lanes must be padded, tied off, or otherwise safely handled by backend wrapper generation and/or the emitted template, and are not exposed as extra user-visible port width.

```
"differential": {
  "source": "DS226-2.6E, Section 2.2 (I/O)",
  "type": "true",
  "io_type": "LVDS25",
  "output": {
    "buffer": {
      "description": "True LVDS differential output buffer",
      "map": { ... }
    },
    "serializer": [
      {
        "description": "4:1 output serializer",
        "ratio": 4,
        "map": { ... }
      },
      {
        "description": "8:1 output serializer",
        "ratio": 8,
        "map": { ... }
      },
      {
        "description": "10:1 output serializer (master+slave cascade)",
        "ratio": 10,
        "map": { ... }
      }
    ]
  },
  "input": {
    "buffer": {
```

```
      "description": "True LVDS differential input buffer",
      "map": { ... }
    },
    "deserializer": [
      {
        "description": "1:4 input deserializer",
        "ratio": 4,
        "map": { ... }
      },
      {
        "description": "1:8 input deserializer",
        "ratio": 8,
        "map": { ... }
      },
      {
        "description": "1:10 input deserializer (master+slave cascade)",
        "ratio": 10,
        "map": { ... }
      }
    ]
  },
  "clock": {
    "buffer": {
      "description": "Differential clock input buffer (global clock network)",
      "map": { ... }
    }
  }
}
```

11. `fixed_pins`

An array of objects describing pins with dedicated (non-user) functions.

Key	Type	Required	Description
`pad`	string	Yes	Internal pad/site name
`name`	string	Yes	Signal name
`pin`	integer	No	QFN pin number (for QFN packages)
`ball`	string	No	BGA ball designator (for BGA/WLCSP packages)
`note`	string	Yes	Description of the pin function

Use `pin` for QFN/QFP packages (integer pin numbers) and `ball` for BGA/WLCSP packages (alphanumeric ball designators). Pad-only entries (no physical pin/ball exposed) omit both.

```
"fixed_pins": [
  { "pad": "IOR32B", "name": "DONE", "note": "Configuration done indicator" },
  { "pad": "RGB0", "name": "RGB0", "pin": 39, "note": "LED driver output" },
```

```
  { "pad": "IOB_32a_SPI_SO", "name": "SPI_SO", "ball": "F1", "note": "SPI configuration flash"
]
```

12. Template Placeholder Syntax

Template strings in `map` arrays use `%%name%%` delimiters to mark substitution points. The compiler replaces these with actual signal names and parameter values during code generation.

12.1 Placeholder Categories

Parameter placeholders reference values from the `parameters` object: - `%%DIV%%`, `%%IDIV%%`, `%%FBDIV%%`, `%%ODIV%%` - `%%DIVR%%`, `%%DIVF%%`, `%%DIVQ%%`, `%%FILTER_RANGE%%` - `%%CLKHF_DIV%%`, `%%DIV_MODE%%`, `%%PLLOUT_SELECT_A%%` - `%%CLKI_DIV%%`, `%%CLKFB_DIV%%`, `%%CLKOP_DIV%%`, `%%CLKOS_DIV%%`, `%%CLKOS2_DIV%%`, `%%CLKOS3_DIV%%` - `%%CLKOUTD_DIV%%`

Derived value placeholders reference values from the `derived` object: - `%%PSDA_SEL%%`

Output signal placeholders reference output names from the `outputs` object: - `%%BASE%%` - Primary clock output - `%%LOCK%%` - Lock indicator - `%%PHASE%%`, `%%DIV%%`, `%%DIV3%%` - Additional clock outputs - `%%CLKOS%%`, `%%CLKOS2%%`, `%%CLKOS3%%` - Secondary outputs (ECP5)

Input signal placeholders reference input names from the `inputs` object: - `%%refclk%%` - Reference clock signal

Feedback wire placeholder references the `feedback_wire` field (see Section 9.9): - `%%<feedback_wire>%%` - Auto-generated feedback wire name (e.g., `%%clkfb%%` resolves to `clkfb_0_0`)

Compiler-generated placeholders: - `%%<INPUT>_mhz%%` - Input clock frequency in MHz (computed by compiler from period) - `%%<INPUT>_period_ns%%` - Input clock period in nanoseconds - `%%instance_idx%%` - Unique instance index for naming - `%%instance%%` - Instance name for differential buffers/serializers

Differential I/O placeholders: - `%%input%%` - Single-ended input signal - `%%output%%` - Single-ended output signal - `%%pin_p%%` - Positive differential pin - `%%pin_n%%` - Negative differential pin - `%%D0%%` through `%%D13%%` - Serializer lane placeholders for the selected primitive ratio; lanes beyond the selected primitive ratio resolve to `1'b0`, and any surplus lanes beyond the logical user-visible width must be padded or tied off by backend wrapper generation and/or the emitted template - `%%Q0%%` through `%%Q13%%` - Deserializer lane placeholders for the selected primitive ratio; lanes beyond the selected primitive ratio resolve to `1'b0`, and any surplus lanes beyond the logical user-visible width must be padded, discarded, or otherwise handled by backend wrapper generation and/or the emitted template - `%%fclk%%` - Fast clock for serializer/deserializer - `%%pclk%%` - Parallel clock for serializer/deserializer - `%%reset%%` - Reset signal for serializer/deserializer - `%%data_width%%` - Serialization width (integer, e.g., `10`) - `%%shiftin1%%`, `%%shiftin2%%` - Cascade shift input wire names (for master+slave templates) - `%%shiftout1%%`, `%%shiftout2%%` - Cascade shift output wire names (for master+slave templates)

12.2 Placeholder Rules

1. All placeholders use double-percent delimiters: `%%name%%`
2. Placeholder names are case-sensitive

3. String-typed parameters are substituted as-is; the template must include quotes if needed
4. Integer-typed parameters are substituted as decimal numbers; the compiler rejects decimal values (e.g., `5.0`) for integer parameters with a `CLOCK_GEN_PARAM_TYPE_MISMATCH` error
5. Double-typed parameters accept both integer and decimal values (e.g., `5` or `5.125`)
6. Signal placeholders are replaced with the actual wire/port name
7. For differential input pins, signal placeholders automatically resolve to the buffered wire (e.g., `%%REF_CLK%%` resolves to `jz_diff_SCLK` instead of `SCLK`)
8. In RTLIL templates, floating-point parameter values must use `parameter real` with quoted string values (e.g., `parameter real \\PHASE "0.0"`)

13. Expression Syntax

Expressions in `frequency_mhz.expr`, `phase_deg.expr`, and `derived.expr` fields use a simple arithmetic expression language.

13.1 Operands

- **Numeric literals**: Integer or floating-point (e.g., `1000.0`, `3`, `45`)
- **Parameter references**: Names from the `parameters` object (e.g., `DIVF`, `ODIV`)
- **Derived value references**: Names from the `derived` object (e.g., `FVCO`)
- **Input references**: `refclk_period_ns` (reference clock period in nanoseconds, derived from the input clock)

13.2 Operators

Operator	Description	Example
`+`	Addition	`DIVF + 1`
`-`	Subtraction	`DIVR + 1`
`*`	Multiplication	`(FBDIV + 1) * ODIV`
`/`	Division	`FVCO / ODIV`
`<<`	Left shift	`1 << DIVQ`

13.3 Functions

Function	Description	Example
`toString`	Convert value to string in given base/width	`toString(PHASESEL * 2, BIN, 4)`

The `toString` function takes three arguments: 1. Value expression 2. Base format (`BIN` for binary) 3. Minimum width (zero-padded)

13.4 Evaluation

Expressions are evaluated left-to-right with standard arithmetic precedence. Parentheses override precedence. All arithmetic is performed in floating-point. The special value `refclk_period_ns` is computed as `1000.0 / refclk_frequency_mhz`.

14. Versioning and Compatibility

When adding new fields: - New optional fields may be added to any object without a version bump - New required fields or changes to existing field semantics require a version bump - Chip data files should be validated against this specification when created or modified

When adding new chips: - Follow the exact schema defined in this document - All `source` fields must reference specific datasheet sections for traceability - Numeric values (resource counts, frequencies, capacities) must match the vendor datasheet